

0915 Answer | 树上的递归究竟传什么

5 道完整原题 · 0914 的扫描状态 · 能力账本

树形只是外壳：每道题分别要传深度、分支数量、括号边界、终止标记或中序前驱。讲解已核原题，限时独立作答未验。

考场起手

题	草稿第一行	检查边界
01	根深度 0；叶子才累加 $\text{weight} \times \text{depth}$	根本身是叶子
02	k 条子边： $\text{km} = \text{N} - 1$	高度从 1 算
03	子树输出时在运算符外包括号	一元减只有一子树
04	编码树遇终点标记后不能再向下	某码是另一码前缀
05	中序遍历每个非 -1 ，与前一个真结点比较	空位不是数值 -1

01 | 2014-41 : WPL 累加的是叶子的深度

41. (13 分) 二叉树的带权路径长度 (WPL) 是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T , 采用二叉链表存储, 结点结构如下:

left	weight	right
------	--------	-------

其中叶结点的 `weight` 域保存该结点的非负权值。设 `root` 为指向 T 的根结点的指针, 请设计求 T 的 WPL 的算法, 要求:

- 1) 给出算法的基本设计思想。
- 2) 使用 C 或 C++ 语言, 给出二叉树结点的数据类型定义。
- 3) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

(1) 思路。根到根的边数为 0; 沿边下探时深度加 1; 只有左右孩子均空的叶子贡献 $\text{weight} \times \text{depth}$, 非叶结点的权值虽存在但不计 WPL。

(2) 结点定义。

```
typedef struct Node {
    struct Node *left;
    int weight;
    struct Node *right;
} Node;
```

(3) 实现。

```
long long wpl(const Node *p, int depth) {
    if (!p) return 0;
    if (!p->left && !p->right) return (long long)p->weight*depth;
    return wpl(p->left, depth+1)+wpl(p->right, depth+1);
}
/* 从 wpl(root,0) 调用 */
```

每结点访问一次, 时间 $O(N)$; 递归栈 $O(h)$, 链状树最坏 $O(N)$ 。根为唯一叶子时深度 0, WPL 为 0。

为什么不能在入结点时就累加

题目明说所有叶的带权路径长度之和; 内部结点权虽非负, 不参加求和。若按层数从 1 乘权, 所有叶子都会多算一个权重。

02 | 2016-42 : 正则 k 叉树的叶子和高度边界

42. 如果一棵非空 k ($k \geq 2$) 叉树 T 中每个非叶结点都有 k 个孩子, 则称 T 为正则 k 叉树。请回答下列问题并给出推导过程。

- (1) 若 T 有 m 个非叶结点, 则 T 中的叶结点有多少个?
- (2) 若 T 的高度为 h (单结点的树 $h=1$), 则 T 的结点数最多为多少个? 最少为多少个?

(1) 已知 m 个非叶结点。每个非叶结点发出 k 条边, 共 km ; 树总边数是 $N-1$ 。所以 $N=km+1$, 叶数 $N-m=(k-1)m+1$ 。不必预先假设每一层都填满。

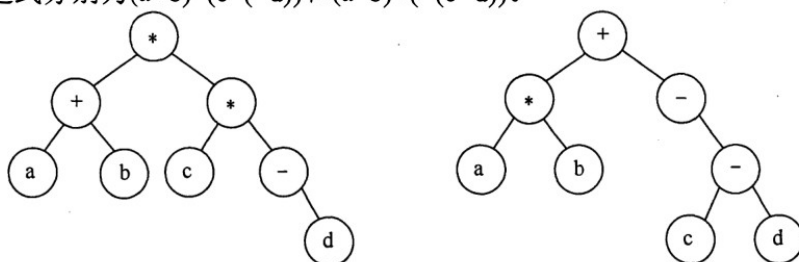
(2) 高度 h 。最多每层全展开，结点数 $1+k+\dots+k^{(h-1)}=(k^h-1)/(k-1)$ 。最少则在前 $h-1$ 层各只留一个内部结点延长主干，每个内部结点依然必须生 k 个孩子： $1+k(h-1)$ 个结点。 $h=1$ 时两端都为1； $h=3$ 、 $k=2$ 的最少形状为根和下一层各有一个内部结点，共5个。

挡住两个直觉错法

最少不是 h 个（那是一条单链，违反每个非叶都有 k 孩子）；最多不是 k^h （那只是下一层的满层规模）。从边数下手可立即拿到第（1）问的分。

03 | 2017-41：树的中序遍历必须保护运算顺序

41. (15 分) 请设计一个算法，将给定的表达式树（二叉树）转换为等价的中缀表达式（通过括号反映操作符的计算次序）并输出。例如，当下列两棵表达式树作为算法的输入时，输出的等价中缀表达式分别为 $(a+b)*(c*(-d))$ 和 $(a*b)+(-(c-d))$ 。



二叉树结点定义如下：

```
typedef struct node{
    char data[10];           //存储操作数或操作符
    struct node *left, *right;
```

```
} BTree;
```

要求：

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

(1) 思路。普通中序只输出符号会把 $a+b$ 接着 $*$ 误读为 $a+b*c$ 。每个运算符子树先输出左括号，递归左子树、运算符、右子树，再输出右括号；叶子直接打印操作数。一元减仅有右孩子时，括号内先打印 $-$ 再打印孩子。

(2) 代码。

```
#include <stdio.h>
typedef struct node {
    char data[10];
    struct node *left,*right;
} BTree;
void printExpr(const BTree *p) {
    if (!p) return;
    if (!p->left && !p->right) { printf("%s",p->data); return; }
    putchar('(');
    if (p->left) printExpr(p->left);
    printf("%s",p->data);           // 一元减在唯一右子树之前
    if (p->right) printExpr(p->right);
```

```
    putchar(')');  
}
```

外层多一对括号不影响等价性：例一为 $((a+b)*(c*(-d)))$ ，例二为 $((a*b)+(-(c-d)))$ 。若要求与题面示例的打印完全一致，只需在根层省去最外括号，内层继续保持； $O(N)$ 时间、 $O(h)$ 栈空间。

一元和二元减怎样辨认

观察孩子数量，不只看字符 $-$ ：图一减结点仅右孩子 d ，输出 $(-d)$ ；图二右侧外减结点仅右孩子为另一个二元减，输出 $(-(c-d))$ 。若树约定唯一孩子在左边，需按数据定义调整分支，而非把右孩子假装为左操作数。

04 | 2020-42：前缀码作为一棵带终点的二叉树

42. (10 分) 若任一字符的编码都不是其他字符编码的前缀，则称这种编码具有前缀特性。

现有某字符集（字符个数 ≥ 2 ）的不等长编码，每个字符的编码均为二进制的 0、1 序列，最长为 L 位，且具有前缀特性。请回答下列问题：

- 1) 哪种数据结构适宜保存上述具有前缀特性的不等长编码？
- 2) 基于你所设计的数据结构，简述从 0/1 串到字符串的译码过程。
- 3) 简述判定某字符集的不等长编码是否具有前缀特性的过程。

(1) 结构。用二叉字典树 (Trie)：每位 0 走左边、1 走右边；字符保存在码字路径终点。最长码长 L ，则根到终点的深度不超过 L 。至少两个字符的前缀码不应把某字符放在根。

(2) 译码。从根读位并移动；到带字符的终点就输出该字符，指针回根继续读。若边不存在或输入结束时停在非终点，这串输入不是该码集可完整译出的串。前缀性质保证抵达终点即完成当前码字，不需猜是否继续向下。

(3) 判定。逐个插入码字：若走位途中遇到已经标为终点的结点，旧码是新码的前缀；到新码终点时若已有字符或已有后代，分别是重复编码或新码为旧码前缀。出现任一种冲突便不具备前缀特性；全插完无冲突才合格。处理所有位的时间 $O(\text{码字总长度})$ ，树空间同阶；译码时间 $O(\text{输入位数})$ 。

为何不能只比相同长度的码

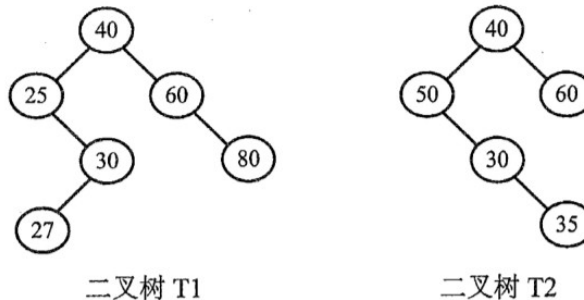
0 与 01 长度不同，读到 0 时已无法知道该停还是继续；根到 0 的结点同时作“终点”和“内结点”就是冲突信号。不同字符也不能共用完全相同的码字。

05 | 2022-41：顺序存储树验证 BST

41. (13 分) 已知非空二叉树 T 的结点值均为正整数，采用顺序存储方式保存，数据结构定义如下：

```
typedef struct { // MAX_SIZE 为已定义常量
    int SqBiTNode[MAX_SIZE]; // 保存二叉树结点值的数组
    int ElemNum; // 实际占用的数组元素个数
} SqBiTree;
```

T 中不存在的结点在数组 $SqBiTNode$ 中用 -1 表示。例如，对于下图所示的两棵非空二叉树 $T1$ 和 $T2$,



$T1$ 的存储结果如下：

$T1.SqBiTNode$

40	25	60	-1	30	-1	80	-1	-1	27		
----	----	----	----	----	----	----	----	----	----	--	--

$T1.ElemNum = 10$

$T2$ 的存储结果如下：

$T2.SqBiTNode$

40	50	60	-1	30	-1	-1	-1	-1	-1	35	
----	----	----	----	----	----	----	----	----	----	----	--

$T2.ElemNum = 11$

请设计一个尽可能高效的算法，判定一棵采用这种方式存储的二叉树是否为二叉搜索树，若是，则返回 `true`，否则，返回 `false`。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

(1) 思路。数组下标 i 的左右孩子分别为 $2i+1$ 、 $2i+2$ ； -1 表示没有结点，即使其下标小于 $ElemNum$ 也要跳过。二叉搜索树中序遍历真实结点应严格递增（题给结点值均为正整数，本解采用无重复键的 BST 约定）。只保留前一真结点值即可。

(2) 代码。

```
typedef struct { int SqBiTNode[MAX_SIZE]; int ElemNum; } SqBiTree;
int check(const SqBiTree *T, size_t i, int *prev, int *have) {
    if (i >= (size_t)T->ElemNum || T->SqBiTNode[i] == -1) return 1;
    if (!check(T, 2*i+1, prev, have)) return 0;
    if (*have && T->SqBiTNode[i] <= *prev) return 0;
    *prev=T->SqBiTNode[i]; *have=1;
    return check(T, 2*i+2, prev, have);
}
int isBST(const SqBiTree *T) {
    int prev=0, have=0;
    return T && T->ElemNum>0 && check(T, 0, &prev, &have);
}
```

```
}
```

T1 的中序为 25, 27, 30, 40, 60, 80, 返回 true; T2 左子树含 50, 根为 40, 中序出现 50, 30, ..., 40, 返回 false。实际访问 ElemNum 范围内下标, 每个真实结点一次, 时间 $O(\text{ElemNum})$ 上界; 额外递归栈 $O(h)$ (顺序存储的最大有效深度)。

为何不能只比较父子

某个结点可能小于自己的父亲, 却大于更上层的祖先, 违反整棵左子树都小于根的条件。中序严格递增将所有祖先约束一次检查完。数组空位 -1 不是可比较的负值, 先过滤再比较。

延迟练习

遮住解答给这五题各写一句“递归/遍历必须带着什么”: 深度、边数、括号、终点、前驱。随后独立完成代码和边界例, 按实际用时记熟练度; 讲解完成不等于通关。